IE 675b: Machine Learning

Assignment 2: Logistic Regression (HWS 2026)

Prof. Dr. Rainer Gemulla
University of Mannheim
Chair of Practical Computer Science I

Task	Maximum	Points
1	4	.
2	6	.
3	6	.
4	6	.
Total	22	.

Rules for Submission

You must carefully follow the rules stated below for this assignment. An assignment that violates any of these rules will be graded as FAIL.

a) Submission

(i) Hand-in your solution via ILIAS until the date specified there. This is a hard deadline.

(ii) We strongly recommend to work on this assignment in pairs—i.e, with one (and only one) additional student—and then hand in a pair submission. Both students must submit exactly the same contents.

(iii) Solution files (such as notebooks and the solution PDF) must clearly state the name and matriculation number of both students right at the beginning.

b) Code

(i) You must submit all relevant code files (e.g., Python scripts) that we require to run your code and to replicate your results. In particular, you must submit all Python scripts that we provided to you.

(ii) You must include a PDF of every Jupyter notebook in which the most relevant cells are executed (i.e., make sure to disable excessively verbose cells that inflate the page count).

(iii) It suffices to fill out the "holes" that are marked in the code fragments.

c) Solution PDF

(i) You must provide your answers by filling out the provided solution LaTeX file. You may add any LaTeX packages that you absolutely need.

(ii) Your solution must be self-contained. i.e. all plots and figures you are referring to in the text must be in the corresponding answer box, alongside said text.

(iii) You must submit a single PDF file (do not submit LaTeX source code). All answers must be provided *inside* the boxes (i.e., no resizing of boxes, no overflow, no additional pages, no modification of font (size), ...) specified therein.

(iv) Name your solution PDF as follows:

```
a0[assignment number]-[ilias name 1]-[ilias name 2].pdf
```

For instance: `a01-aturing-jvneumann.pdf`

d) AI usage

(i) You must fill out and sign the AI declaration at the end of the solution file following the provided instructions.

(ii) You take responsibility for all solutions that you hand in. You must know and understand all solutions and be able to explain and defend them.

(iii) Excessive AI usage (such as handing in pure AI solutions) is considered cheating and will be reported to the examination board. Potential measures include failing the assignment, failing the course, or ex-matriculation.

Preliminaries: Spambase Dataset

In this assignment, we are using a dataset on email spam detection, provided in the file `data/spamData.mat`. You can find more information about this dataset at `https://archive.ics.uci.edu/ml/datasets/spambase`.

The dataset is derived[1] from 4601 emails, each being labeled as no-spam (0) or spam (1). Each example has 57 features:

- 48 word features, each indicating the frequency percentage of a word in the email (e.g., "business", "free", "george").
- 6 character features, each indicating the frequency percentage of a character in the email (for `{[}!$#`).
- 3 features on length statistics of consecutive upper case letters (e.g., "HELLO" gives 5): on for average length, one for maximimum length, and one for the sum of all such all-capitalised sequences in an email.

The dataset is split into a training set (3065 examples) and a test set (1536 examples). We provide code to load the data and provide all feature names.

Task 1: Dataset Statistics — 4 Points

Explore and preprocess the dataset. (4 points)

a) Look at the kernel density plot (code provided) of all features and discuss what you see (or don't see).

b) Normalize the data using z-scores, i.e., normalize each feature to mean 0 and variance 1. Normalize both training and test data. In particular, think about how test data should be normalized.

Make sure to stick to the variable names provided in the code fragements. From now on, **we will exclusively work with the normalized data**.

c) Redo the kernel density plot on the normalized data. What changed? Is there anything that "sticks out"?

[1] I.e., a basis function that extracts the features given above has been applied to the emails already.

Task 2: Empirical Risk Minimization — 6 Points

Consider empirical risk minimization with log loss for logistic regression.

a) Show analytically that if we use a bias term, then rescaling (multiply by constant) and/or shifting (add a constant) features leads to ERM estimates with the same empirical risk. Why do you think we computed z-scores then?

b) Complete the methods for computing the empirical risk and its gradient in Python. We **do not use a bias term** throughout.

c) Implement gradient descent in Python, using the framework provided to you.

   Optional. Can you implement each gradient descent epoch using only vectorized operations (no loops)?

d) Implement stochastic gradient descent.

e) Explore the behavior of both methods for the parameters provided to you. Discuss!

Task 3: Prediction — 6 Points

a) Complete the `predict` and `classify` methods for the predicted spam probability and predicted class label, respectively.

b) Explore the models that you fit in the previous task and discuss.

c) Study the composition of the weight vector: which features are important, which are not? Is this intuitive?

Task 4: Regularized Risk Minimization — 6 Points

Consider regularized risk minimization with log loss and L2 regularization for logistic regression.

a) Implement gradient descent for RRM with hyperparameter λ. You can reuse the methods of your solution for ERM.

b) Study the effect of regularization on the result by varying the value of λ. Consider at least the empirical risk, the average log loss on the test data, and the prediction accuracy. Are these results surprising to you?

c) Study the composition of the weight vector for varying choices of λ (try very large values). Try to explain what you saw in the task above.

Task 5: Exploration (optional)

Explore variants of preprocessing and logistic regression further. Suggestions include:

- Try gradient descent on the original data without using z-scores.
- Add a bias feature (make sure that you do not "normalize" it).
- Try to reduce the training set size and compare ERM and RRM estimation (ideally using cross-validation).
- Run a logistic regression method from some existing library. Do you get the same results?
- Experiment with different gradient-based optimizers. To do so, we provide a PyTorch implementation, which allows you to quickly change the optimizer. For a list of optimizers, see here.
- Try the Newton method (we derived gradient and Hessian in the exercises).